LLM 应用框架 前置知识

> 这份文档为零基础读者准备。如果你已经熟悉 LangChain、LlamaIndex、Agent、RAG（Retrieval-Augmented Generation，检索增强生成）Pipeline 等概念，可以直接翻阅主报告。

> 🎯 **读完这篇你能**：根据应用场景选择 Pipeline / DAG / Agent 架构模式，判断何时引入 RAG 和 ReAct 推理循环，并为一个简单需求选型合适的 LLM 框架。

一、LLM 应用框架到底是什么？用一句话说清楚

LLM 应用框架 = 让开发者不用重复造轮子的工具箱，用来快速搭建基于大模型的应用。

如果直接调 LLM API（比如 ChatGPT），你只能做一问一答。但一个真实的 AI 应用需要：

- 让模型能查资料（RAG）
- 让模型能用工具（搜索、计算器、数据库）
- 让模型能记住对话历史
- 让模型能自主规划任务（Agent）

这些能力的每一条都需要写大量代码。LLM 应用框架把这些"基础设施"封装好了，开发者只需要几行代码就能组合出复杂的 AI 应用。

二、核心概念

Pipeline / Chain（流水线）

最基本的概念——把多个步骤串起来。比如：

```
用户提问 → 检索相关文档 → 把问题和文档一起发给 LLM → 输出回答
```

类比：就像工厂里的流水线——原材料（用户问题）从一头进去，经过多个工序（检索→增强→生成），成品（答案）从另一头出来。每一步都是固定的，顺序不变。

框架让你用声明式的方式定义这个流程，不需要手写每一步的代码。

DAG（Directed Acyclic Graph，有向无环图）

当流程不是简单的一条线，而是有分支、有并行的复杂结构时，就用到 DAG。

类比：Pipeline 像做菜时按顺序放调料（先放油→再放葱→再放菜），DAG 像同时煮着米饭、炒着菜、炖着汤——三条线并行，最后一起上桌。

比如一个知识库机器人可以"同时搜索文档库和数据库 → 把两个结果合并 → 发给 LLM"，这就是一个 DAG。现代 LLM 框架的内部编排引擎大多基于 DAG 设计。

Agent（智能体）

一个"会自己决定做什么"的 LLM 程序。Agent 不只是执行固定流程，而是在循环中：

```
接收任务 → 推理（该做什么）→ 执行（调用工具）→ 观察结果 → 再推理 → ...
```

类比：Pipeline 像自动售货机——投币（输入），掉出可乐（输出），固定流程。Agent 像一个人类店员——你说了"我要喝点什么"，他会自己判断你应该喝可乐还是咖啡，去拿，回来给你，不够凉还会去加冰。

Agent 框架处理这个循环的编排逻辑（状态管理、容错、决策机制）。

ReAct（推理+行动）

Agent 最核心的工作模式。由 2022 年 Yao et al. 提出（Yao et al., 2022），核心思想是"边思考边行动"：

```
思考"我需要查什么" → 调用搜索工具 → 看到结果 → 思考下一步 → 调用另一个工具 → ...
```

ReAct 锁定了几乎所有现代 Agent 框架（LangChain、AutoGen、CrewAI）的设计方向。一句话说清楚：**没有 ReAct，Agent 只会闭门造车；有了 ReAct，Agent 能一边推理一边验证。**

Tool / Plugin（工具）

LLM 需要调用外部能力——搜索引擎、数据库、计算器、API。框架提供统一的方式来定义和调用这些工具。

类比：就像给你一个助理，不仅给他一张嘴（LLM），还给了他书包里的各种工具——计算器、电话本、文件柜。他平时用不上，但需要时伸手就能拿到。

Memory（记忆）

多轮对话需要记忆。框架提供：

- 短期记忆（当前会话的对话历史）——类比：你记得住刚才聊了啥
- 长期记忆（跨会话的关键信息）——类比：隔一周回来，还记得你的名字和偏好

RAG Pipeline（检索增强生成）

从文档中检索相关内容并注入 prompt。框架处理：文档解析 → 分块 → 向量化 → 检索 → 重排序 → 生成的完整链路。

Token（词元）

Token 是大模型处理文本的最小单位。简单理解——模型不读"字"，读"词元"。"你好世界"=4个 token。API 按 token 计费，框架的优化目标很多就是"用更少的 token 完成任务"。

MCP（Model Context Protocol）

Anthropic 在 2024 年底推出的开放协议（Anthropic, 2024），统一 Agent 与外部工具的连接方式，被称为"AI 的 USB-C"。主流框架（LangChain、Dify）都已原生支持 MCP，使其成为 Agent-工具连接的标准化接口。

LangGraph（状态图框架）

LangChain 的 Agent 编排引擎，采用**有向图状态机**架构取代了早期的 Chain 模型。Agent 的每一步都是图上的一个节点，节点之间有状态传递。LangGraph 是 2024-2026 年 Agent 编排领域的事实标准，能处理复杂的循环、分支和条件跳转。

三、主流框架一句话定位

框架	一句话定位	谁在用
LangChain	通用编排框架，700+ 集成（LangChain Inc.），生态最大	AI 工程师、Agent 开发者
LangGraph	Agent 状态图引擎，LangChain 的 Agent 核心	需要复杂 Agent 的团队
LlamaIndex	检索和数据索引最强，适合企业知识库	知识库、文档处理团队
Dify	可视化拖拽平台，非技术人员也能用	产品经理、业务团队、GCC
Haystack	模块化、可审计，适合受监管行业	金融、医疗、政府
MAF (原 Semantic Kernel)	.NET 生态，企业合规级，M365 深度集成	.NET / Azure 用户
DSPy	自动优化 prompt 和 few-shot，编译器式框架	追求极致 prompt 质量的团队
CrewAI	多 Agent 角色扮演，简单协作	Agent 原型的快速验证
AutoGen	微软多 Agent 对话框架	多 Agent 研究和原型
Coze / 扣子	字节跳动低代码 Agent 平台	国内非技术用户

四、LangChain vs LlamaIndex——最关键的区别

这两个是最容易被混淆的框架，简单区分：

	LangChain	LlamaIndex
起源	做"应用编排"	做"数据索引"
最擅长	Agent 工作流、工具调用	文档解析、高级检索策略
核心抽象	Chain / Graph / Agent	Index / Retriever / Document
典型用法	定义 Agent 调用哪些工具	构建复杂文档的知识库

实践中常组合使用：**LlamaIndex 处理文档和检索，LangChain 编排工作流。**

五、Dify 的可视化 ≠ 不作为

Dify 在 2026 年达到 140k GitHub Stars（LangGenius）（超过了 LangChain），说明**低代码/无代码不是"玩具"**。它的增长来自：

- 业务人员可以直接搭建 AI 应用，不需要等开发资源
- 政府和央企场景（GCC）对可视化配置有刚需
- 企业版支持自托管（私有部署）+ MCP 协议

六、一张图总结

```
                LLM 应用框架全景
                       |
        ┌──────────────┼──────────────┐
        │              │              │
    编排型          检索/RAG型      可视化型
        │              │              │
  LangChain       LlamaIndex        Dify
  LangGraph        Haystack         (拖拽)
  Semantic
  Kernel
        │              │              │
        └──────────────┼──────────────┘
                       |
                  编译优化型
                       |
                    DSPy

 2026 年趋势:
 ────────────────
 Agent 是主战场 → LangGraph 领跑
 低代码爆发 → Dify 140k stars 超越 LangChain (LangGenius)
 MCP 协议 → 框架都在适配
 组合式堆栈 → LlamaIndex 做检索 + LangChain 做编排
```

参考来源

- **ReAct: Synergizing Reasoning and Acting in Language Models** (Yao et al., 2022) — 提出"边思考边行动"范式，锁定了几乎所有现代 Agent 框架的设计方向 — https://arxiv.org/abs/2210.03629
- **DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines** (Khattab et al., Stanford, 2023) — 编译器式框架，自动优化 prompt 和 few-shot 示例 — https://arxiv.org/abs/2310.03714
- **LangChain** (LangChain Inc.) — 通用 LLM 编排框架，700+ 集成，LangGraph 是 Agent 编排领域事实标准 — https://www.langchain.com/
- **LlamaIndex** (LlamaIndex Inc.) — 以数据索引和检索为核心的框架，擅长文档解析与高级检索策略 — https://www.llamaindex.ai/
- **Dify** (LangGenius) — 可视化拖拽式 LLM 应用平台，2026 年 140k GitHub Stars，支持自托管和 MCP 协议 — https://github.com/langgenius/dify
- **Model Context Protocol Specification** (Anthropic, 2024) — 统一 Agent 与外部工具连接方式的开放协议，主流框架均已原生支持 — https://modelcontextprotocol.io/